WIKIPEDIA
25 years of the free encyclopedia

Magik (programming language)

Magik is an object-oriented programming language that supports multiple inheritance and polymorphism, and it is dynamically typed. It was designed and implemented in 1989 by Arthur Chance of Smallworld Systems Ltd. as part of Smallworld Geographical Information System (GIS). Following Smallworld's acquisition in 2000, Magik is now provided by GE Energy, still as part of its Smallworld technology platform.

Magik (Inspirational Magik) was originally introduced in 1990 and has been improved and updated over the years. Its current version is 5.2.

In July 2012, Magik developers announced that they were in the process of porting Magik language on the Java virtual machine. The successful porting was confirmed by Oracle Corporation in November of the same year.[1]

Similarities with Smalltalk

Magik itself shares some similarities with Smalltalk in terms of its language features and its architecture: the Magik language is compiled into byte codes interpreted by the Magik virtual machine. The Magik virtual machine is available on several platforms including Microsoft Windows, various flavours of Unix and Linux.

Magik is console based and code can be modified on the fly even when an application is running. The console can also be used to execute Magik code and to see the results.

Compiled code is stored in a single file called an image file. Each image file holds the compiled byte-codes and the state of the session (for example variable values) when the image was last saved.

Language features

Comments

Magik uses the **#** token to mark sections of code as comments:

```
# This is a comment.
```

Assignments

Magik uses the **<<** operator to make assignments:

```
a << 1.234
b << b + a
```

```
c << "foo" + "bar" # Concat strings
```

For clarity, this notation is read as "a becomes 1.234" or "b becomes b plus a". This terminology separates assignment from comparison.

Magik also supports a compressed variation of this operator that works in a similar way to those found in C:

```
b +<< a # Equivalent to b << b + a
```

To print a variable you can use the following command

```
a << "hello"
write(a)
```

Symbols

As well as conventional data types such as integers, floats and strings Magik also implements symbols. Symbols are a special token data type that are used extensively throughout Magik to uniquely identify objects. They are represented by a colon followed by a string of characters. Symbols can be escaped using the vertical bar character. For example:

```
a << :hello  # whenever :hello is encountered, it is the same instance
b << :|hello world|
```

Dynamic typing

Magik variables are not typed as they are in say C# and can reference different objects at runtime. Everything in Magik is an object (there is no distinction between objects and primitive types such as integers):

```
a << 1.2     # a floating point number is assigned to variable 'a'
a << "1.2"   # later, a string is assigned to variable 'a'
```

Objects

Objects are implemented in Magik using exemplars. Exemplars have similarities to classes in other programming languages such as Java, but with important differences. Magik supports multiple inheritance, and mixins (which implement functionality with no data). New instances are made by cloning an existing instance (which will typically be the exemplar but does not have to be).

New exemplars are created using the statement `def_slotted_exemplar()`, for example:

```
def_slotted_exemplar(:my_object,
{
  {:slot_a, 34},
  {:slot_b, "hello"}
}, {:parent_object_a, :parent_object_b})
```

This code fragment will define a new exemplar called `my_object` that has two slots (or fields) called `slot_a` (pre-initialised to 34) and `slot_b` (pre-initialised to "hello") that inherits from two existing exemplars called `parent_object_a` and `parent_object_b`.

Comparison

Magik implements all usual logical operators (=, <, <=, >, >=, ~=/<>) for comparison, as well as a few unusual ones. The `_is` and `_isnt` operators are used for comparing specific instances of objects, or object references rather than values.

For example:

```
   a << "hello"
   b << "hello"

   a = b # returns True (_true) because the values of a and b are equal
   a _is b # returns False (_false) because a is not the same instance as b

   a << "hello"
   b << a
   a = b # returns True (_true) because the values of a and b are equal
   a _is b # returns True (_true) because b was assigned the specific instance of the same object as a,
 rather than the value of a.
```

Methods

Methods are defined on exemplars using the statements `_method` and `_endmethod`:

```
   _method my_object.my_method(a, b)
     _return a + b
   _endmethod
```

It is convention to supply two methods `new()` (to create a new instance) and `init()` (to initialise an instance).

```
   # New method
   _method person.new(name, age)
     _return _clone.init(name, age)
   _endmethod

   # Initialise method.
   _private _method person.init(name, age)
      # Call the parent implementation.
      _super.init(name, age)
      # Initialise the slots.
      .name << name
      .age << age
     _return _self
   _endmethod
```

The `_clone` creates a physical copy of the `person` object. The `_super` statement allows objects to invoke an implementation of a method on the parent exemplar. Objects can reference themselves using the `_self` statement. An object's slots are accessed and assigned using a dot notation.

Methods that are not part of the public interface of the object can be marked private using the `_private` statement. Private methods can only be called by `_self`, `_super` and `_clone`.

Optional arguments can be declared using the `_optional` statement. Optional arguments that are not passed are assigned by Magik to the special object `_unset` (the equivalent of null). The `_gather` statement can be used to declare a list of optional arguments.

```
   _method my_object.my_method(_gather values)
   _endmethod
```

Iteration

In Magik the `_while`, `_for`, `_over`, `_loop` and `_endloop` statements allow iteration.

```
_block
        _local s << 0
        _local i << 0
        _while i <= 100
        _loop
                s +<< i
                i +<< 1
        _endloop
>> s
_endblock
```

Here, the _while is combined with _loop and _endloop.

```
   _method my_object.my_method(_gather values)
     total << 0.0
     _for a _over values.elements()
     _loop
        total +<< a
     _endloop
     _return total
   _endmethod

   m << my_object.new()
   x << m.my_method(1.0, 2, 3.0, 4) # x = 10.0
```

Here values.elements() is an iterator which helps to iterate the values.

In Magik generator methods are called iterator methods. New iterator methods can be defined using the `_iter` and `_loopbody` statements:

```
   _iter _method my_object.even_elements()
     _for a _over _self.elements()
     _loop
       _if a.even? _is _true
       _then
          _loopbody(a)
       _endif
     _endloop
   _endmethod
```

Procedures

Magik also supports functions called procedures. Procedures are also objects and are declared using the `_proc` and `_endproc` statements. Procedures are assigned to variables which may then be invoked:

```
   my_procedure << _proc @my_procedure(a, b, c)
     _return a + b + c
```

```
  _endproc

  x << my_procedure(1, 2, 3) # x = 6
```

Regular expression

Magik supports // regular expression syntax:

```
_if /Hello\,\s(\w)+!/.matches?("Hello, Magik!") _then
    write("Got a match!")
_endif
```

and to capture groups in Regex:

```
/sw([0-9]+)-([0-9]+).*/.replace_all("sw65456-324sss", "$1") # "65456"
/sw([0-9]+)-([0-9]+).*/.replace_all("sw65456-324sss", "$2") # "324"
```

HTTP library

Magik supports making HTTP or HTTPS requests via http library, see below examples:

```
magikhttp << http.new()
magikhttp.url("https://www.google.com").get()
magikhttp.url("https://www.google.com").post({"User-agent", "Bot"}, "some data")
```

Language quirks

Because Magik was originally developed in England, methods in the core smallworld libraries are spelled using British English. For example:

```
  Use "initialise", not "initialize".
```

Collections

Like other programming language Magik too has collections. They include the following:

- Simple Vector
- Rope - a dynamic array
- Hash Table
- Property List
- Equality set
- Bags

Hello World example

The following is an example of the Hello world program written in Magik:

```
 write("Hello World!")
```

References

1. Jim Connors (2012-11-05). "Sprinkle Some Magik on that Java Virtual Machine" (https://blogs.oracle.com/jtc/entry/sprinkle_some_magik_on_that). Retrieved 2012-11-06. "*With this new capability GE Energy has succeeded in hosting their Magik environment on top of the Java Virtual Machine*"

External links

- Smallworld Product Suite Technology (http://www.ge-energy.com/products_and_services/products/geospatial_systems_and_mobile_workforce/index.jsp)
- MDT - Magik Development Tools (http://www.mdt.net/) IDE for GE Smallworld GIS developers
- Open Source (SourceForge) (https://sourceforge.net/projects/magikcmpnts)
- Language forum post on Magik (http://lambda-the-ultimate.org/classic/message5839.html#5886)
- Technical Paper No. 5 - An Overview of Smallworld Magik (https://web.archive.org/web/20110114022720/http://cfis.savagexi.com:80/pages/technical_paper_5)
- GE Smallworld, Emacs Extensions for Magik developers (http://www.magikemacs.com/page/6/) Archived (https://web.archive.org/web/20160425011048/http://www.magikemacs.com/page/6/) 2016-04-25 at the Wayback Machine
- Visual Studio Code extension for Smallworld Magik programming language. (https://marketplace.visualstudio.com/items?itemName=siamz.smallworld-magik)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Magik_(programming_language)&oldid=1364524401"